

Lecture 4: Neural Networks and Backpropagation

선형 분류기 사이에 비선형 활성화 함수를 넣어 쌓은 것이 신경망이며 가중치를 학습하는 데 필요한 기울기는 계산 그래프 위에서 **역전파**로 계산한다.

Neural Network

Neural networks: 2 layers

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

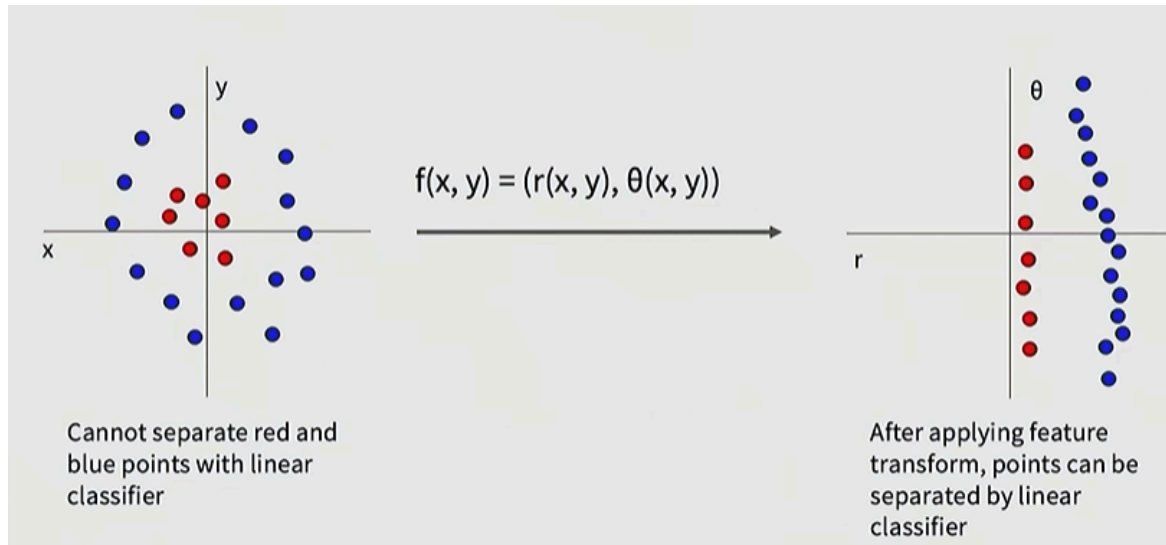
$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

- 선형 분류기 : $f = Wx$ (선형 함수 하나로 이루어진 가장 기본적인 형태)
- 2층 신경망 : $f = W_2 \max(0, W_1 x)$
 - D : 입력 차원(특징의 개수), H : 은닉층의 뉴런 수, C : 클래스 수(출력의 개수)
 - $W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$
 - $\max$ 연산은 W_1 과 W_2 에 의한 두 선형 변환 사이에 **비선형성**을 부여하며, 신경망 구성에서 핵심적인 역할을 한다.

Q. 활성화 함수(예 : $\max$) 없이 신경망을 구성하면 어떻게 되는가?

A. $f = W_2 W_1 x$ 가 되며, 이는 하나의 행렬 $W_3 = W_2 W_1$ 로 대체될 수 있다. 즉, 층을 아무리 쌓아도 결국 하나의 선형 함수에 불과하다.

Non-linearity

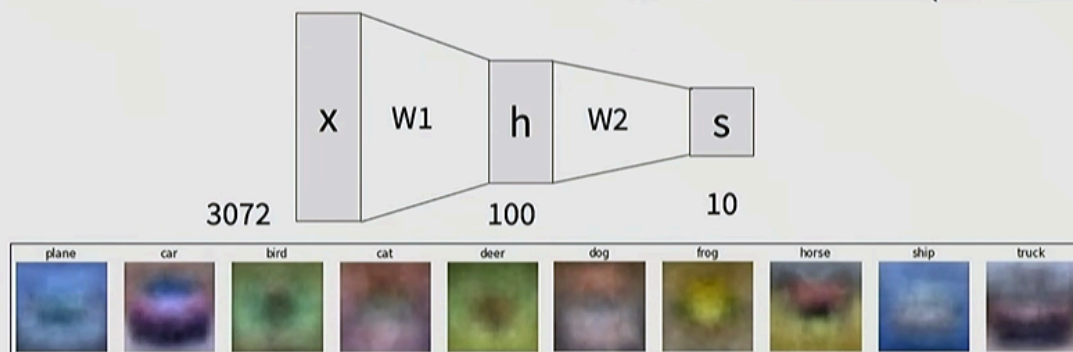


- 실제 문제 중 상당수는 하나의 직선으로 샘플을 구분할 수 없다.
- 위 그림은 비선형 변환을 통해 데이터를 원래 공간에서 새로운 특징 공간으로 옮기면, 그 공간에서는 선형 분류가 가능해진다는 것을 보여준다.
- **MLP(다층 퍼셉트론)** : 선형 층과 활성화 함수를 번갈아 쌓은 신경망(완전 연결 신경망). layer을 추가하여 더 깊은 신경망을 구성할 수 있다.

Neural networks: learning 100s of templates

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



Learn 100 templates instead of 10.

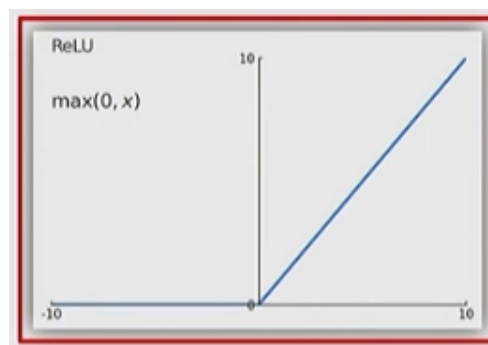
Share templates between classes

Stanford

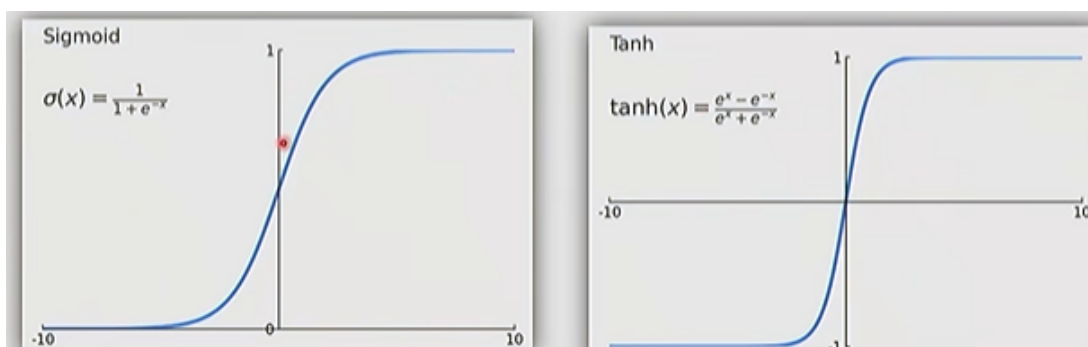
: 신경망이 가중치를 통해 학습한 템플릿의 시각화

- 선형 분류기는 클래스당 하나씩, 총 10개의 템플릿만 학습할 수 있다.
- 은닉층(h)에 뉴런 100개를 두면 100개의 템플릿을 학습할 수 있으며, 객체 전체가 아니라 **객체의 일부**에 대한 템플릿도 만들 수 있다.
- 예를 들어 새, 고양이, 사슴, 개는 모두 눈을 가지고 있으므로, '눈' 템플릿은 여러 클래스가 공유할 수 있다.

활성화 함수 (Activation Functions)



- **ReLU** : $\max(0, x)$. 신경망에서 가장 널리 사용되는 활성화 함수이다.
 - (-) : 입력이 음수이면 출력과 기울기가 모두 0이 되어, 더 이상 학습되지 않는 뉴런 (dead neuron)이 생길 수 있다.
 - 이를 보완하기 위해 음수 구간에서도 작은 값을 출력하는 **Leaky ReLU**, **ELU** 등을 사용할 수 있다.
- **GELU**, **SiLU** 등 최근의 변형 함수도 있으며, 주로 Transformer 계열 모델에서 사용된다.



- **Sigmoid, Tanh** : 출력을 좁은 범위로 압축하기 때문에, 입력의 절댓값이 크면 기울기가 0에 가까워지는 **기울기 소실** 문제가 있다.

- 따라서 신경망의 중간층보다는 주로 출력층에서 사용한다.

Q. 새로운 문제에 어떤 활성화 함수를 사용할지 어떻게 결정하는가?

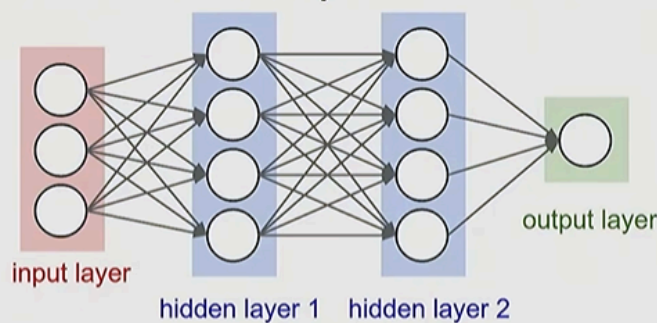
A. 대부분 경험적으로 결정한다. 보통 ReLU에서 출발하거나, CNN·Transformer 등 해당 구조에서 이미 검증된 표준 활성화 함수를 사용한다.

활성화 함수의 공통 조건

- 비선형이어야 한다.
- 경사하강법으로 학습하므로 미분 가능해야 한다.
- 출력이 0을 중심으로 하고 매끄러울수록 학습이 빠르게 수렴한다.

ex. 3층 신경망

Example feed-forward computation of a neural network



```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

: 3층 완전 연결 신경망의 구조와 순전파 코드

- 각 W 는 이전 층과 다음 층 사이의 가중치 정의
- sigmoid 사용, 은닉층 값은 $h_1 = f(W_1x + b_1)$, $h_2 = f(W_2h_1 + b_2)$ 로 계산
- 출력층은 W_3 과 마지막 은닉층 값의 곱으로 계산

신경망 구현

1. 네트워크 정의

```
N, D_in, H, D_out = 64, 1000, 100, 10
x, y = randn(N, D_in), randn(N, D_out)
w1, w2 = randn(D_in, H), randn(H, D_out)
```

Define the network

N : 샘플 수 / D_in : 입력 차원 / D_out : 출력 차원 / H : 은닉층의 뉴런 수.

입력-정답 데이터를 만들고 가중치를 무작위로 초기화한다.

2. 순전파(forward pass)

```
for t in range(2000):
    h = 1 / (1 + np.exp(-x.dot(w1)))
    y_pred = h.dot(w2)
    loss = np.square(y_pred - y).sum()
    print(t, loss)
```

Forward pass

가중치를 층별로 적용하여 예측값을 만들고 손실 함수로 손실값을 계산한다.

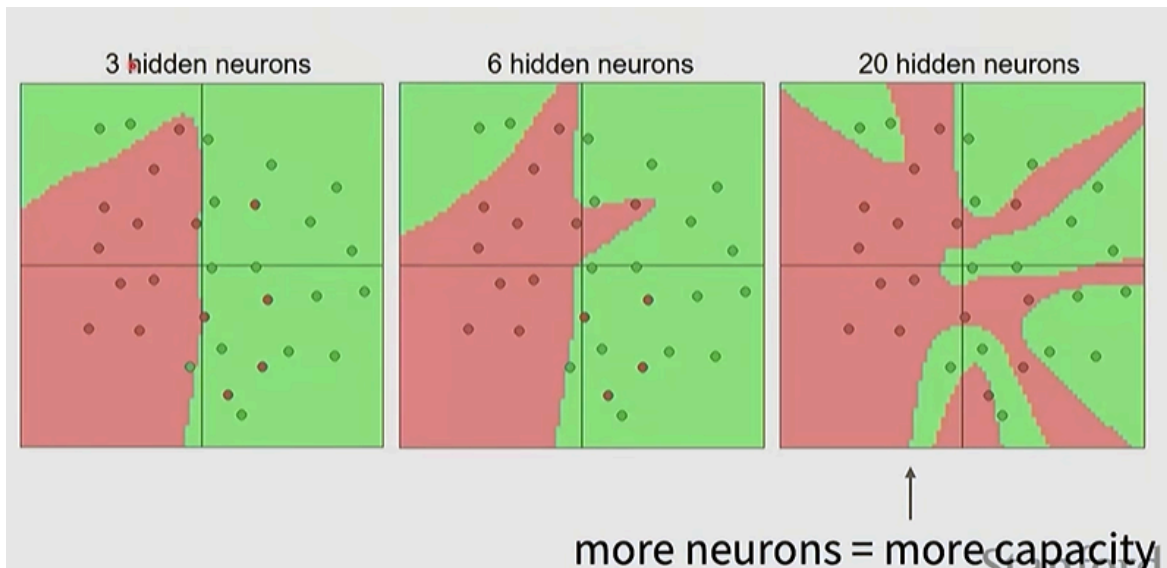
3. 역전파 및 최적화

```
grad_y_pred = 2.0 * (y_pred - y)
grad_w2 = h.T.dot(grad_y_pred)
grad_h = grad_y_pred.dot(w2.T)
grad_w1 = x.T.dot(grad_h * h * (1 - h))
```

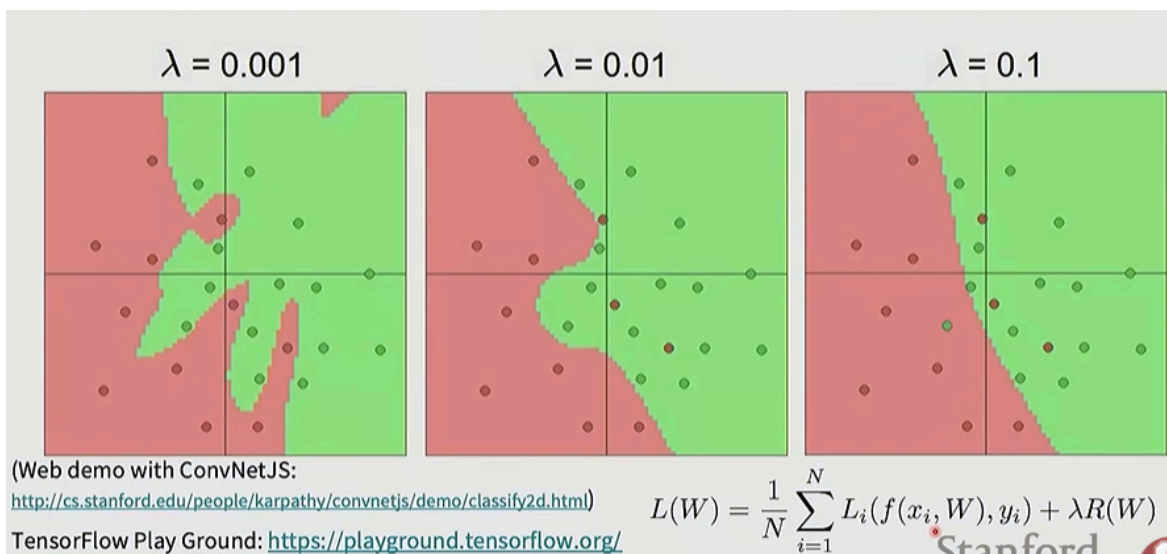
Calculate the analytical gradients

해석적 기울기를 계산하고, 이를 이용한 경사하강법으로 W_1 , W_2 를 갱신한다. 이 기울기를 계산하는 방법이 역전파.

신경망의 크기와 정규화



- 은닉층의 뉴런 수가 많을수록 더 복잡한 함수를 학습할 수 있어 두 클래스를 더 정교하게 분리한다.
- 그러나 용량(capacity)이 과도하면 학습 데이터를 암기하여 **과적합**이 발생할 수 있다.



- 신경망의 크기를 정규화 수단으로 사용하지 않는다. 과적합을 막기 위해서는 신경망을 줄이는 대신 **정규화를 강하게** 적용한다.
- 위 그림에서 정규화 강도 λ 가 클수록 결정 경계가 부드러워진다.
- 다만 λ 가 지나치게 크면 가중치가 과도하게 제약되어 경계가 단순해지고, 오히려 **과소적합**이 발생한다.

과적합을 막는 두 가지 방법

1. 신경망을 작게 만든다.
2. 충분히 큰 신경망을 사용하고 정규화를 적용한다.

실제로는 2번을 사용한다. 즉, 주로 조정하는 대상은 신경망의 크기가 아니라 **정규화 하이퍼파라미터(λ)**이다.

Backpropagation

문제 : 기울기를 어떻게 계산할 것인가

Problem: How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{Hinge Loss on predictions}$$

$$R(W) = \sum_k W_k^2 \quad \text{Regularization}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad \text{Total loss: data loss + regularization}$$

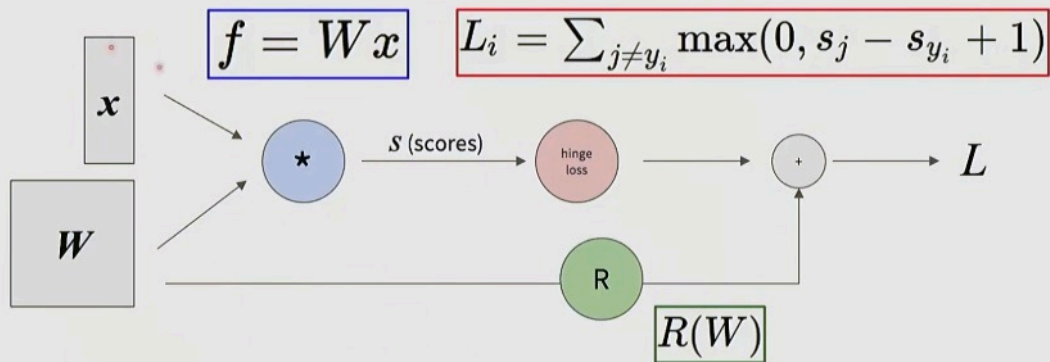
If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

Stanford

- 전체 손실 = 데이터 손실(hinge loss, softmax 등) + 정규화 항 $R(W)$
- 경사하강법을 위해서는 $\partial L / \partial W_1, \partial L / \partial W_2$ 가 필요하다.
- 손으로 직접 유도하는 방식의 한계
 - 행렬 미분 계산이 번거롭다.
 - 손실 함수를 조금만 바꿔도 처음부터 다시 유도해야 한다.
 - 모델이 복잡해지면 사실상 불가능하다.

Computational Graph (계산 그래프)

Better Idea: Computational graphs + Backpropagation



Stanford

- 모델의 모든 연산을 노드로 표현하고, 입력과 파라미터에서 손실까지의 계산 과정을 그래프로 나타낸 것.
- 이 그래프 위에서 역전파를 수행하면 복잡한 모델이라도 기울기를 자동으로 계산할 수 있다.

ex. $f(x, y, z) = (x + y)z$

Backpropagation: a simple example

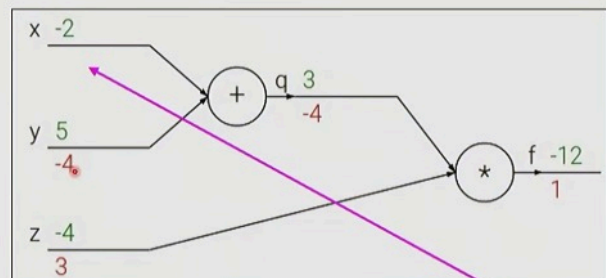
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream gradient Local gradient

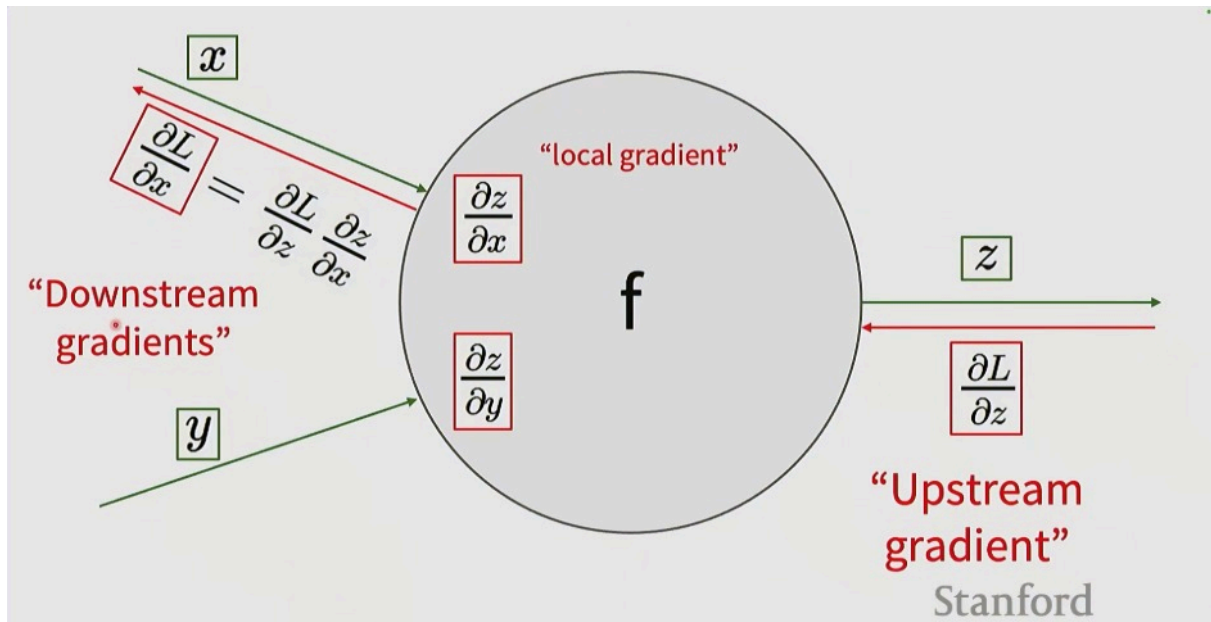
Stanford

$x = -2, y = 5, z = -4$ 일 때

- 순전파 : $q = x + y = 3, f = qz = -12$

- 역전파 : 출력에서 시작하여($\partial f / \partial f = 1$) 입력 방향으로 거슬러 올라간다.
 - $\partial f / \partial z = q = 3, \partial f / \partial q = z = -4$
 - x, y 는 f 와 직접 연결되지 않으므로 **연쇄 법칙(chain rule)**을 적용한다 : $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x} = -4 \times 1 = -4, \partial f / \partial y = -4$

노드 단위의 역전파



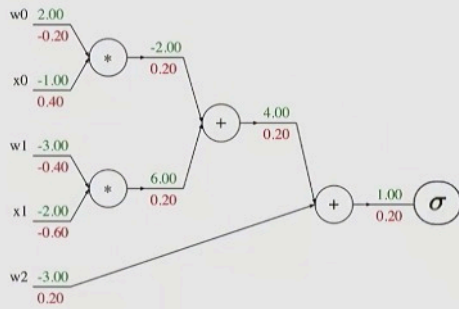
용어	의미
상류 기울기 (upstream gradient)	뒤쪽(출력 쪽)에서 전달된 기울기 $\partial L / \partial z$
국소 기울기 (local gradient)	노드의 출력에 대한 입력의 기울기 $\partial z / \partial x$
하류 기울기 (downstream gradient)	상류 기울기 $\times$ 국소 기울기 $= \partial L / \partial x$

- 하류 기울기는 앞쪽 노드의 상류 기울기가 되며, 이 과정을 입력까지 재귀적으로 반복한다.

각 노드는 **자신의 국소 기울기만 알면** 된다. 전체 함수를 미분할 필요 없이 국소적인 계산만으로 모든 파라미터의 기울기를 얻을 수 있으며, 이것이 역전파가 수백 개의 층에도 적용될 수 있는 이유이다.

기울기 흐름의 패턴

Backprop Implementation: “Flat” code



Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
    s0 = w0 * x0
    s1 = w1 * x1
    s2 = s0 + s1
    s3 = s2 + w2
    L = sigmoid(s3)
```

Backward pass:
Compute grads

```
grad_L = 1.0
grad_s3 = grad_L * (1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

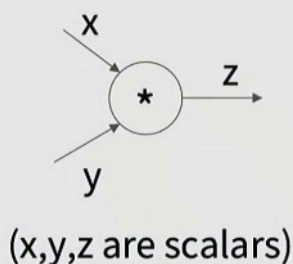
Stanford

게이트	역할	동작
add	기울기 분배기 (distributor)	상류 기울기를 모든 입력에 그대로 전달
mul	교환 곱셈기 (swap multiplier)	상류 기울기 × 상대편 입력값
copy	기울기 합산기 (adder)	여러 갈래에서 온 기울기를 더함
max	기울기 라우터 (router)	최댓값이었던 입력에만 기울기 전달, 나머지는 0

모듈화된 구현 : forward / backward API

Modularized implementation: forward / backward API

Gate / Node / Function object: Actual PyTorch code



```
class Multiply(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y):
        ctx.save_for_backward(x, y)
        z = x * y
        return z
    @staticmethod
    def backward(ctx, grad_z):
        x, y = ctx.saved_tensors
        grad_x = y * grad_z # dz/dx * dL/dz
        grad_y = x * grad_z # dz/dy * dL/dz
        return grad_x, grad_y
```

Need to cache some
values for use in
backward

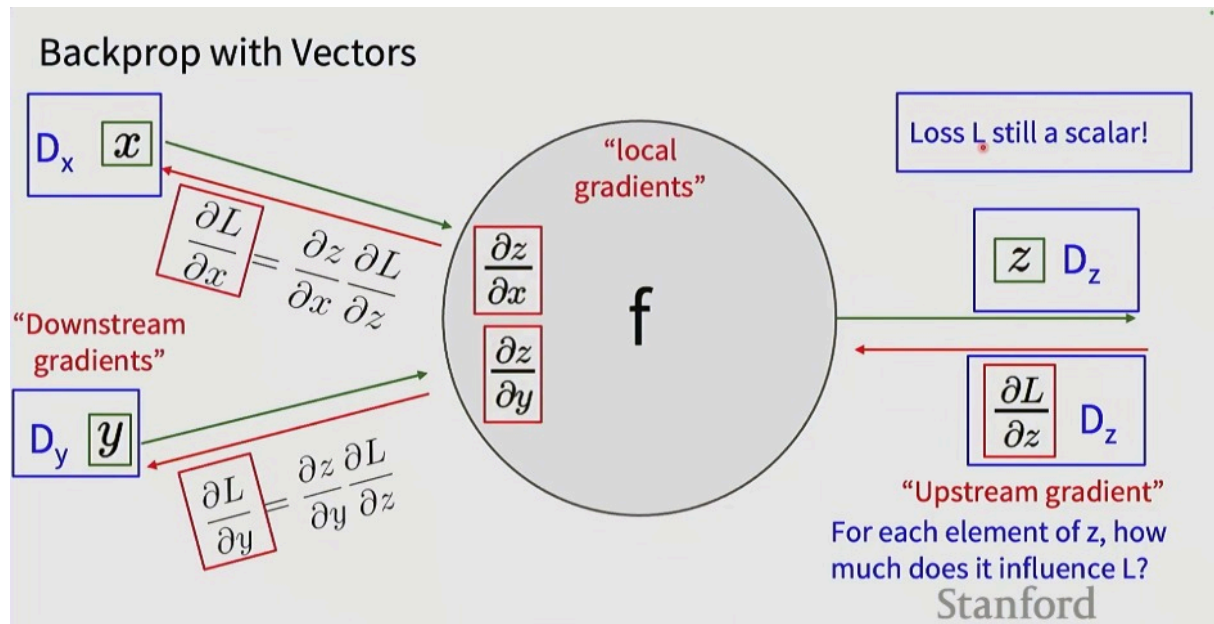
Upstream
gradient

Multiply upstream
and local gradients

Stanford

- 각 연산을 **forward**(출력 계산 + 역전파에 필요한 입력 저장)와 **backward**(상류 기울기 × 국소 기울기)를 가진 모듈로 정의한다.
- PyTorch의 autograd 연산자도 이와 동일한 구조로 구현되어 있다.

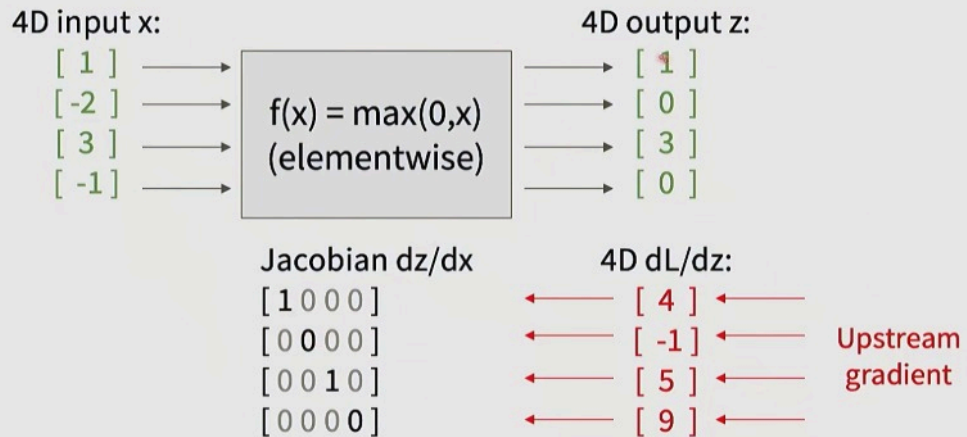
벡터와 행렬에서의 역전파



입력 → 출력	미분의 형태
스칼라 → 스칼라	스칼라
벡터 → 스칼라	기울기 벡터 (각 원소가 변할 때 출력이 변하는 정도)
벡터 → 벡터	야코비안(Jacobian) 행렬 (각 입력 원소가 각 출력 원소에 미치는 영향)

- 손실 L 은 항상 스칼라이므로, 어떤 변수에 대한 기울기는 그 변수와 같은 크기를 가진다.
- 국소 기울기는 야코비안 행렬이 되고, 하류 기울기 = 야코비안 × 상류 기울기로 계산한다.

Backprop with Vectors

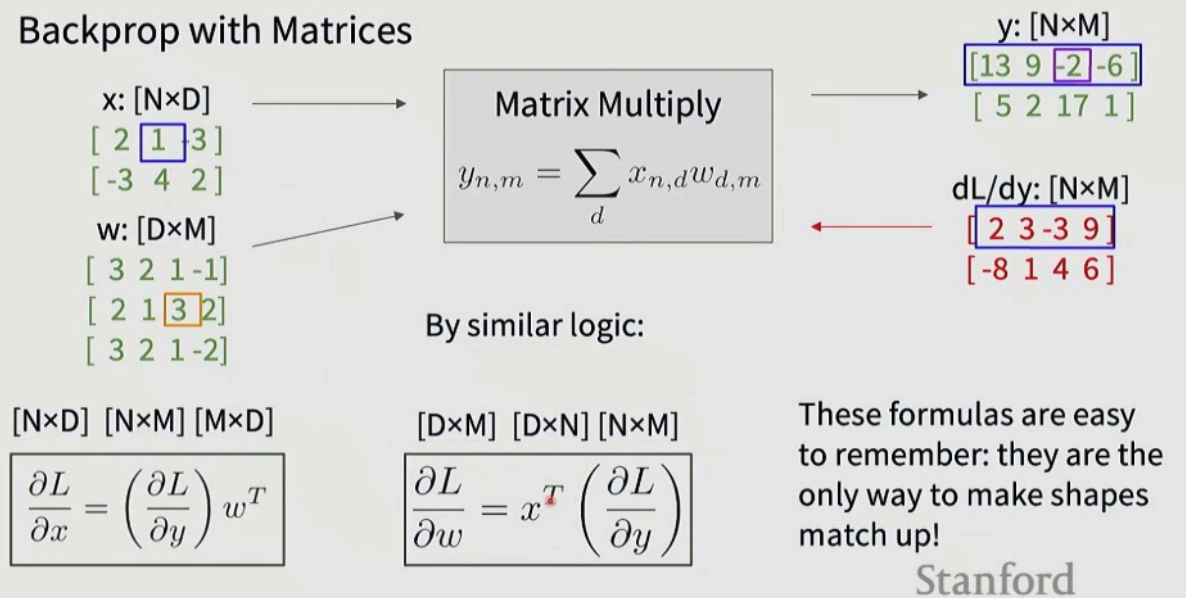


Stanford

위 그림 : $\text{ReLU}(\max(0, x))$ 의 역전파

- 원소별(elementwise) 연산이므로 야코비안은 대각 성분만 0 또는 1인 **희소 행렬**이다.
- 실제로는 이 행렬을 만들지 않고, 입력이 양수였던 위치의 기울기만 그대로 통과시킨다.

Backprop with Matrices



Stanford

위 그림 : 행렬 곱 $y = xW$ 의 역전파

- 야코비안을 직접 만들면 매우 크다. (예 : $N = 64, D = M = 4096$ 이면 약 256GB)
- 대신 각 원소가 출력의 어느 부분에 영향을 주는지 분석하면 다음과 같이 간단한 행렬 곱으로 정리된다.

- $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} W^T, \frac{\partial L}{\partial W} = x^T \frac{\partial L}{\partial y}$
- 곱셈 게이트의 '교환' 원리와 같으며, 결과의 크기가 원래 변수의 크기와 같아야 한다는 점으로 계산할 수 있다.